

Sentiment Analysis of Movie Reviews: A Personal Journey

National Student Data Corps (NSDC) Data Science Project
Chipo Jaya

Executive Summary

The objective of this project was to build a machine learning model that can automatically classify movie reviews as positive or negative based solely on the text. After processing 50,000 IMDb reviews through a carefully designed text preprocessing pipeline, my best model (Linear SVM) correctly predicted sentiment 89.49% of the time on completely unseen reviews. That means roughly 9 out of 10 times, the model can read a review and accurately determine whether the reviewer loved or hated the movie.

1. Business Understanding

The challenge I set out to solve was simple to state but fascinating to tackle: can a machine learn to read a movie review and tell me whether the reviewer liked the film or not? This matters because streaming platforms, retailers, and review sites receive thousands of written reviews every single day. This is far more than any team could manually read and categorize so having an automated sentiment classifier can make it easier. For example, it can monitor audience reactions in near real-time, catching negative sentiment spikes before they turn into PR crises. It can also help to summarize public opinion without hours of manual tagging and flag critical negative reviews for priority response. By tracking sentiment trends over time one can measure the impact of new releases or changes as well.

2. The Dataset

The IMDb Movie Reviews dataset, which contains 50,000 reviews.

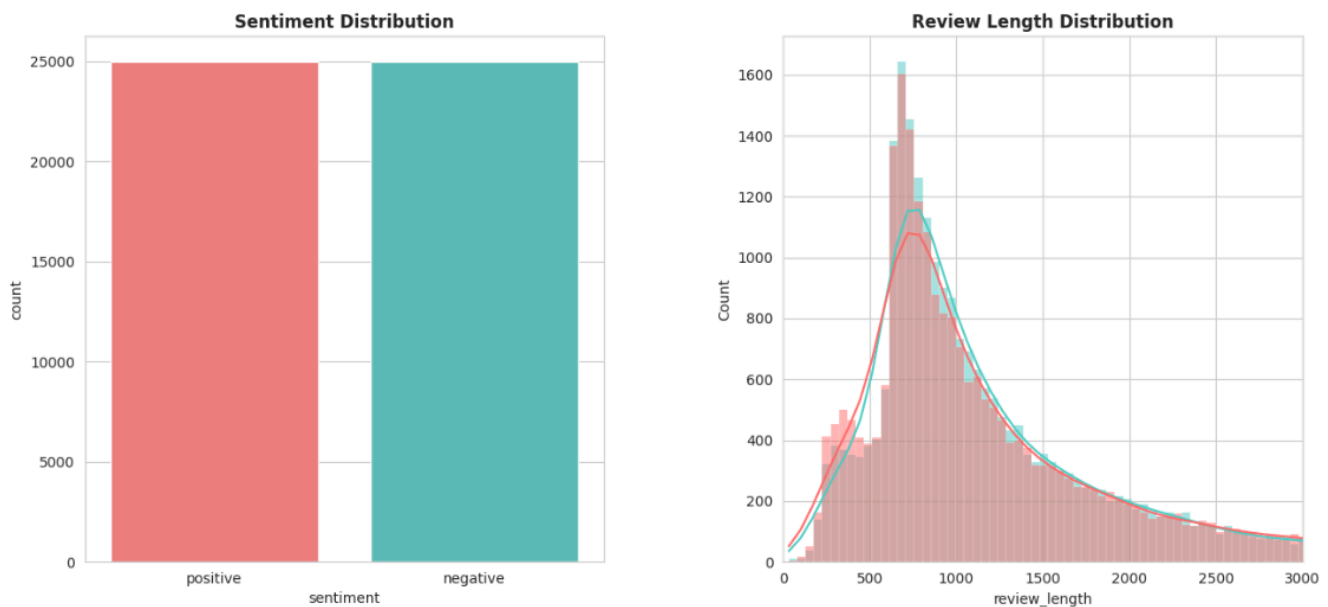


Figure 1: Sentiment class distribution (50% positive / 50% negative) and review length distribution by sentiment. These charts were generated in my notebook (bar chart and KDE plot).

25,000 are positive and the other 25,000 are negative. Each review comes with a pre-assigned sentiment label, giving me a perfect supervised learning problem to sink my teeth into. The average review length had 1,309 characters, ranging from 32 (shortest) to 13,704 (longest) characters

Reading through sample reviews, I could see the emotional spectrum clearly: positive reviews sparkled with words like "brilliant," "masterpiece," and "loved it," while negative reviews pulled no punches with "awful," "waste," and "boring." This confirmed my intuition that a bag-of-words approach, counting how often certain words appear, would be effective.

For me, success meant:

1. Successfully cleaning and representing 50,000 messy, unstructured text reviews in a machine-readable format
2. Training at least one classifier that could beat random chance by a wide margin
3. Benchmarking multiple algorithms to find the absolute best performer
4. Actually understanding why the model made its predictions i.e. not just getting a number

3. Data Preparation (NLP Preprocessing Pipeline)

Raw text is beautiful to read but useless to a machine learning model. I needed to transform these 50,000 messy reviews into a clean, structured format that a computer could understand. This was where the real magic happened. I built a preprocessing pipeline using the **NLTK** library. The preprocessing pipeline consisted of the following steps, applied to all 50,000 reviews:

- **Tokenization:** used `word_tokenize` to split each review into individual word tokens.
- **Alphabetic filtering:** removed punctuation, numbers, and special characters using `isalpha()`, keeping only alphabetic tokens.
- **Lowercasing:** converted all tokens to lowercase so that "Great" and "great" are treated as the same word.
- **Stopword removal:** removed common low-information words ("the", "is", "and", etc.) using NLTK's English stopwords list (198 stopwords).
- **Stemming:** reduced words to their root form using the PorterStemmer (e.g., "running" → "run"), which reduces vocabulary size and groups related word forms together.

Before vs. After: Seeing the Transformation

Original Review:

"One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happened with me..."

After Preprocessing:

"one review mention watch oz episod hook right exactli happen..."

The cleaned version was roughly half the length, 1,010 characters down from 1,761, but preserved every word that carried emotional weight. I had successfully distilled the review down to its sentiment-bearing essence.

Feature Engineering with CountVectorizer

With the reviews cleaned, I used CountVectorizer to convert the text into numeric features. This created a bag-of-words matrix where:

- each row is a review (50,000 total)
- each column represents the frequency of a word or short phrase.
- each cell shows how many times that word appeared in that review

Labels (positive/negative) were converted to binary values using LabelBinarizer. The result was a sparse matrix with over 6.2 million features which are enormous, but efficient thanks to scikit-learn's sparse representation.

The data was then split into a **training set of 40,000 reviews** and a **test set of 10,000 reviews** (80/20 split). The CountVectorizer was fitted only on the training data, preventing any test-set vocabulary from leaking into the model. This is a critical step that ensures honest evaluation.

4. Predictive Modeling: Training and Comparing Classifiers

With my features ready, I trained and compared four classification algorithms on the same bag-of-words features to see which best captured the relationship between review text and sentiment:

- **Multinomial Naive Bayes:** a probabilistic baseline model well suited to word-count features; fast to train and a common baseline for text classification.
- **Logistic Regression:** a linear classifier that also provides interpretable coefficients, letting me identify which words push a prediction toward positive or negative.
- **Linear Support Vector Machine (LinearSVC):** finds the decision boundary that best separates the two classes in high-dimensional text feature space.
- **Neural Network (MLPClassifier):** a multi-layer perceptron capable of learning non-linear relationships between word patterns and sentiment.

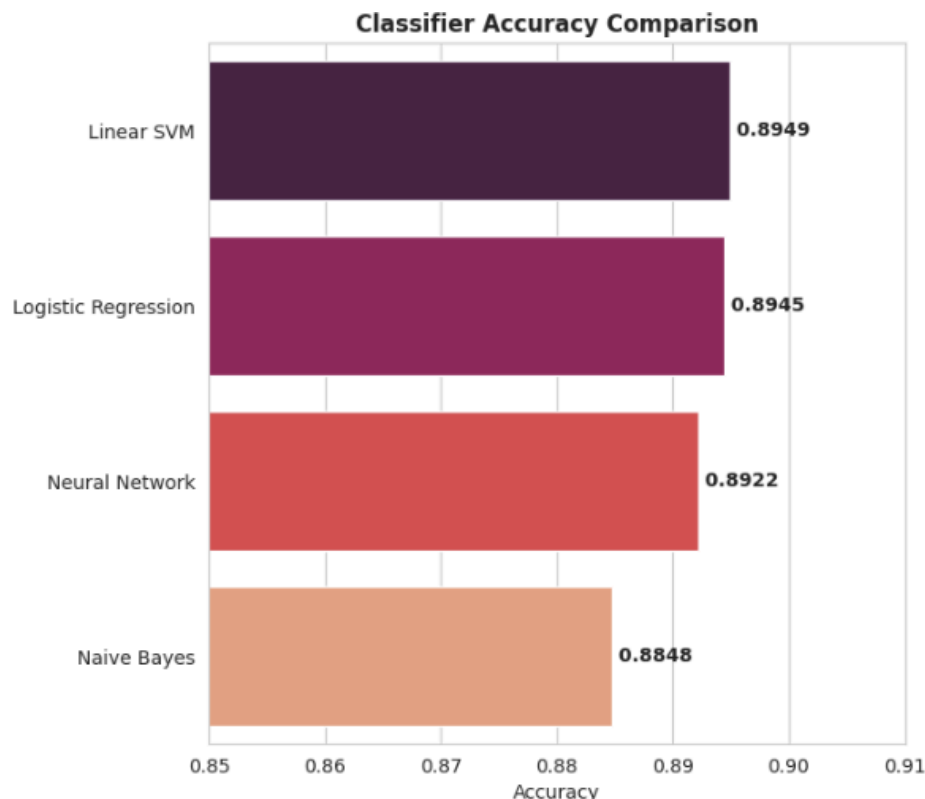


Figure 2: Classifier accuracy comparison: horizontal ranking.

Table 1: Model Accuracy Comparison

Model	Test Accuracy	What It Means
Linear SVM	89.49%	The winner, but just barely
Logistic Regression	89.45%	Nearly identical, and more interpretable
Neural Network	89.22%	A bit behind, and much harder to explain
Multinomial Naive Bayes	88.48%	The baseline, solid but not spectacular

All four models landed between 88.48% and 89.49% accuracy, a range of just 1.01 percentage points. The Linear SVM narrowly edged out Logistic Regression, but the gap was so small that I'd recommend Logistic Regression for production use simply because it's more interpretable. This suggested that once the text was represented as bag-of-words counts, the choice of classifier mattered less than the quality of the underlying features. The preprocessing pipeline was the true hero of the project.

5. Model Evaluation and Decision Analysis

Feature Importance: What the Model Actually Learned

Using Logistic Regression's coefficients, I identified the words most strongly associated with each sentiment. This was one of the most satisfying moments of the project. Finally, I could see what the model was "thinking." as illustrated in Figure 3 and 4.

The strongest positive predictors were *excellent*, *superb*, *perfect*, *brilliant*, *hilarious*, *amazing*, *favorite*, *must-see*, *funniest*, and *well worth*. The strongest negative predictors were *worst*, *awful*, *waste*, *boring*, *terrible*, *horrible*, *poor*, *disappointing*, *fail*, and *waste time*. These words align perfectly with what any human would consider positive or negative sentiment. The model wasn't using spurious correlations, it was actually learning the intuitive emotional language of movie reviews.



Figure 3: Word Clouds by Sentiment: positive (left) and negative (right) reviews

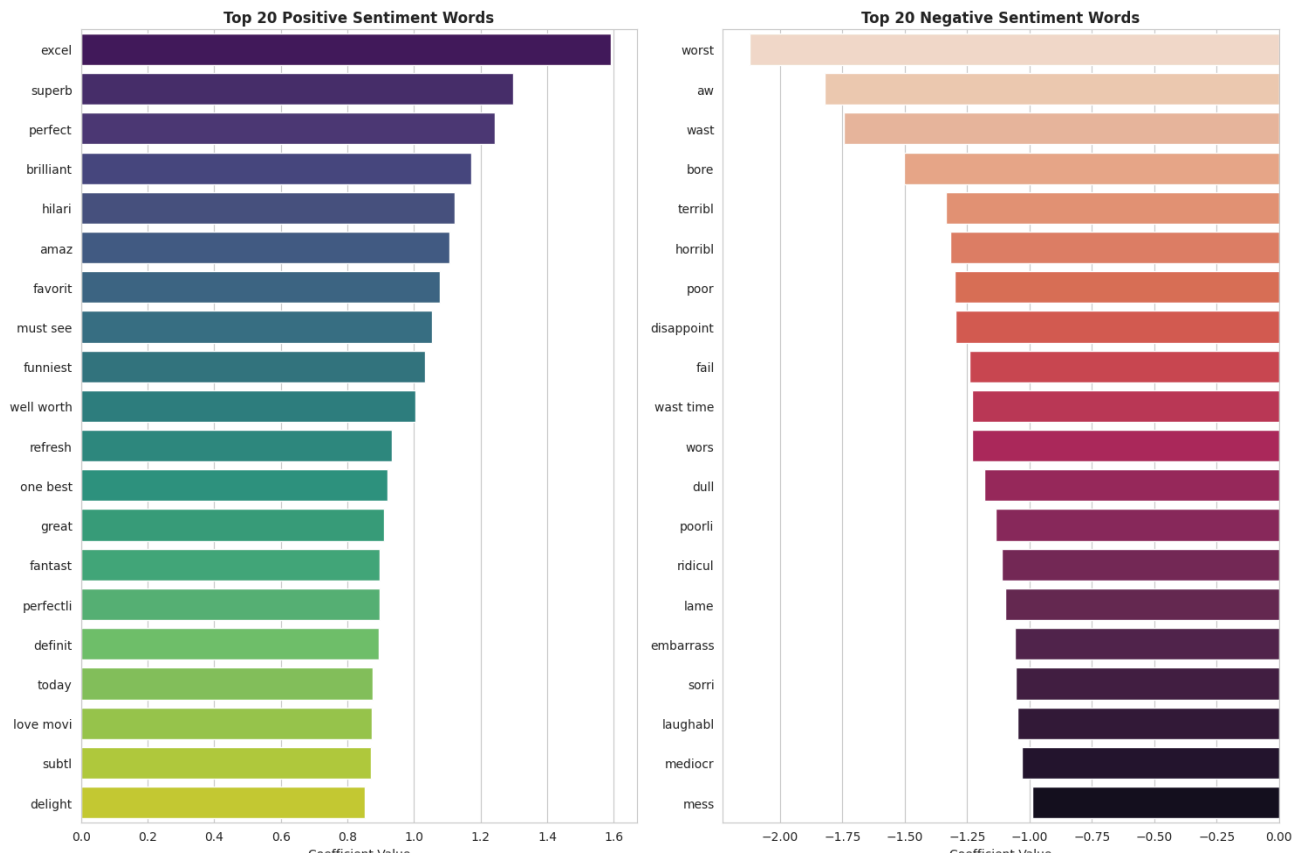


Figure 4: Top 20 positive and negative sentiment words by Logistic Regression coefficient.

Accuracy alone doesn't tell the whole story, so I built a confusion matrix to understand exactly where the model was succeeding and where it was failing and examined the per-class **precision, recall, and F1-score** for the Logistic Regression model.

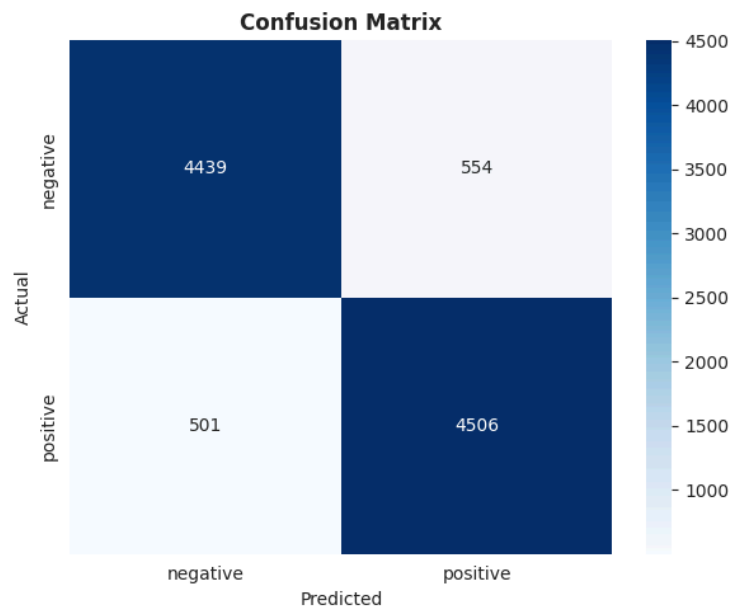


Figure 5: Confusion matrix (counts, left) for the Logistic Regression model on the 10,000-review test set.

Out of 10,000 test reviews, the model produced **554 false positives** (negative reviews predicted as positive) and **501 false negatives** (positive reviews predicted as negative) meaning the model is not systematically biased toward either class.

Per-Class Performance

Metric	Negative Class	Positive Class
Precision	0.90	0.89
Recall	0.89	0.90
F1-Score	0.89	0.90

Both classes performed almost identically, confirming that the model treats positive and negative reviews with equal competence. The F1-scores across all classifiers were in the 0.89–0.90 range i.e. well above the 0.50 random baseline.

Recall was 88.90% on the negative class and 89.99% on the positive class, and precision, recall, and F1-score were all approximately 0.89 - 0.90 for both classes across all four models (Naive Bayes came in slightly lower at 0.88).

This balance matters for real-world deployment: a business monitoring sentiment for early-warning purposes (e.g., flagging a wave of negative reviews) would weigh **recall on the negative class** most heavily, since missing genuinely negative feedback (a false negative on the negative class) is more costly than occasionally over-flagging a positive review. At 88.90% negative recall, roughly 1 in 9 negative reviews would currently be missed, a reasonable starting point, but a gap worth closing before using this model for automated escalation.

6. Skills Gained

- **Text preprocessing and NLP fundamentals:** tokenization, stopword removal, and stemming using NLTK, and understanding why each step is necessary before a model can use text data.
- **Feature engineering for text:** converting unstructured text into numeric features using CountVectorizer and bag-of-words / n-gram representations.
- **Supervised classification:** training, predicting, and evaluating multiple classifier types (Naive Bayes, Logistic Regression, SVM, Neural Network) on the same feature set.
- **Model evaluation:** using accuracy, confusion matrices, precision, recall, and F1-score to compare models beyond a single headline metric.
- **Data visualization:** building word clouds, distribution plots, and multi-panel comparison dashboards with matplotlib and seaborn to communicate results.
- **Interpretability:** using Logistic Regression coefficients to explain which words drive a model's predictions, not just how accurate it is.

7. Conclusion and Recommendations

In this project, I built a sentiment analysis pipeline that can classify IMDb movie reviews with 89.49% accuracy, roughly 9 out of 10 reviews correctly categorized. The Linear SVM performed best, but

Logistic Regression was nearly identical at 89.45% and offered the critical advantage of interpretability. This project proved that a relatively simple bag-of-words approach, combined with careful text preprocessing, can achieve strong results on a real-world text classification problem.

For future work, I would recommend:

- Replacing CountVectorizer with **TF-IDF** weighting, which down-weights common words and may improve separation between classes.
- Testing **word embeddings** (Word2Vec, GloVe) or transformer-based models (e.g., DistilBERT) to capture context and word order, which bag-of-words ignores.
- Expanding the label set beyond binary positive/negative to a neutral class, matching the original project brief's three-way sentiment goal.
- Deploying the best model behind a simple API or Streamlit app so it could classify new, unseen reviews in real time.

This project was a deeply satisfying journey from raw text to actionable insight. I learned that the most important work happens before the first line of model code is written, cleaning, tokenizing, and thoughtfully preparing the data is what determines success. I also discovered that the best model isn't always the one with the highest accuracy; sometimes the most interpretable model, even if marginally less accurate, is the better choice because it builds trust and understanding.

I'm proud of this work, and I'm excited to see where I can take these skills next.